

# MOTHER OF AI — CURATOR ARXIV

Architecture Systemic & Feuille de Route Agentique (Agentic RAG)

## 1. VISION : DU CHATBOT À L'AGENT

L'évolution des systèmes basés sur les LLM s'articule autour de trois stades majeurs de maturité technologique :

| Niveau              | Métaphore                 | Comportement                                                                                           |
|---------------------|---------------------------|--------------------------------------------------------------------------------------------------------|
| Chatbot             | Le perroquet à l'accueil  | Récite des fiches par cœur sans véritable compréhension du contexte.                                   |
| RAG classique       | Le bibliothécaire         | Va chercher les bons livres selon une requête simple et statique.                                      |
| Agent (Agentic RAG) | Le Directeur de Recherche | Planifie des revues de littérature, orchestre des outils, contrôle les résultats et ajuste sa mémoire. |

## 2. MATRICE DES 5 PATTERNS AGENTIQUES

| Pattern                | Rôle système                                 | Composant technique clé                             |
|------------------------|----------------------------------------------|-----------------------------------------------------|
| 1. Tool Use            | Interagir avec des systèmes externes via API | Function calling (JSON typé), FastAPI, LlamaIndex   |
| 2. Reflection          | Évaluation critique et auto-correction       | Boucle Generate/Evaluate, RAGAS, assertions         |
| 3. Plan & Execute      | Décomposition stratégique et replanification | Airflow DAG, LangGraph Planner, Query Decomposition |
| 4. Orchestrator/Worker | Délégation hiérarchique, contexte isolé      | Master Agent + Sub-Agents, Celery/Python Workers    |
| 5. Memory & Context    | Gestion sélective mémoire court/long terme   | OpenSearch, PostgreSQL, Chunking sémantique         |

### 3. ARCHITECTURE TECHNIQUE

#### FLUX GLOBAL DE L'ARCHITECTURE AGENTIQUE

UI (Telegram / Gradio)



FastAPI Orchestrator (Pattern 4)

#### [ Module Agentique ]

LangGraph Agent

- Plan, Replan, Tool Selection
- Interroge **Ollama LLM** (inférence locale)
- Interroge **Hybrid Search** (OpenSearch / BM25 + vecteurs)

#### [ Pipeline Ingestion ]

Airflow Ingestion Pipeline

- Fetch, OCR, batch
- Alimente **PostgreSQL DB** + Index vectoriel

### 4. DÉCLINAISON PAR PATTERN

#### 1. Tool Use — Donner des mains au LLM

Le LLM (Ollama) ne se limite plus à générer du texte : via FastAPI + LlamaIndex, il appelle des outils typés.

| Outil                    | Fonction                               |
|--------------------------|----------------------------------------|
| outil_recherche_hybride  | Interroge OpenSearch (BM25 + vecteurs) |
| outil_filtre_metadonnees | Filtre PostgreSQL (année, auteur...)   |

**Saut agentique (semaine 7) :** Avec LangGraph, le LLM décide quel outil appeler — relance la recherche si le contexte est insuffisant, vérifie un auteur en base si besoin.

#### 2. Reflection — Auto-correction et validation

| Mécanisme        | Rôle                                                                         |
|------------------|------------------------------------------------------------------------------|
| Re-ranking       | Évalue et filtre les chunks avant de les transmettre au LLM (le "goûteur")   |
| RAGAS + Langfuse | Mesure asynchrone : Faithfulness, Context Precision, Answer Relevancy        |
| Fallback OCR     | Si le parsing PDF échoue, bascule automatique sur extraction OCR             |
| User Feedback    | Collecté via Langfuse, alimente l'évaluation globale et le futur fine-tuning |

### 3. Plan & Execute — Décomposer la complexité

#### Airflow DAG (ingestion) :

- **Plan** : Métadonnées → Téléchargement → Extraction → Chunking → Vectorisation
- **Execute** : Chaque étape exécutée séquentiellement
- **Replan** : Papier corrompu → erreur isolée, le reste du batch continue

#### Query Decomposition (semaine 7) — Exemple : "Compare les approches Transformers entre 2022 et 2024"

1. Recherche "Transformers 2022"
2. Recherche "Transformers 2024"
3. Synthèse des deux résultats

### 4. Orchestrator / Worker — Déléguer à l'échelle

| Composant        | Rôle                                                                        |
|------------------|-----------------------------------------------------------------------------|
| FastAPI          | Orchestrateur web — reçoit la requête, délègue de façon async               |
| LangGraph        | Master Agent — délègue à des sub-agents (recherche, synthèse, vérification) |
| Celery / Workers | Exécutent les tâches lourdes (téléchargement arXiv, OCR) sans bloquer l'API |

**Bénéfice clé** : Contexte isolé par worker → pas de saturation du LLM avec des informations inutiles.

### 5. Memory & Context Management — Ne pas saturer

| Type de mémoire        | Implémentation          | Rôle                                                                                            |
|------------------------|-------------------------|-------------------------------------------------------------------------------------------------|
| Working Memory         | Context Builder         | Top-K chunks + Prompt Template. Mémoire immédiate, limitée par la fenêtre de contexte d'Ollama. |
| Long-Term (sémantique) | OpenSearch / PostgreSQL | Millions de chunks vectorisés, chargés sélectivement via Hybrid Search.                         |
| Chunking sémantique    | Découpage ciblé         | Découpage par section / paragraphe pour éviter de saturer le LLM de bruit.                      |
| Episodic Memory        | Historique fil Telegram | Traite les questions de suivi contextualisées.                                                  |

## 5. FEUILLE DE ROUTE

| Phase        | Livrables techniques                                                            | Patterns couverts                                                 |
|--------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Semaines 1-3 | Pipeline Airflow, modélisation PostgreSQL/OpenSearch, endpoints FastAPI de base | Ingénierie classique (aucun pattern agentique)                    |
| Semaines 4-5 | Chunking sémantique, intégration LlamaIndex/Ollama, recherche hybride           | Memory (retrieval) + Tool Use basique                             |
| Semaine 6    | Langfuse, métriques RAGAS, fallback OCR                                         | Reflection                                                        |
| Semaine 7    | LangGraph, décomposition de requêtes, bot Telegram persistant                   | Plan & Execute + Orchestrator/Worker → <b>Agentic RAG complet</b> |

## 6. GRILLE D'ÉVALUATION AVANT PRODUCTION

| # | Pattern             | Critère Go / No-Go                                                                          |
|---|---------------------|---------------------------------------------------------------------------------------------|
| 1 | Tool Use            | Arguments d'appel avec schéma strict et validé (Pydantic / JSON Schema) ?                   |
| 2 | Reflection          | Validation basée sur des assertions/métriques fermées, pas sur une auto-évaluation du LLM ? |
| 3 | Plan & Execute      | Replanification autonome en cas d'échec d'une sous-tâche ?                                  |
| 4 | Orchestrator/Worker | Isolement de contexte par worker — pas de transfert excessif d'info inutile ?               |
| 5 | Memory              | Filtres d'élagage à l'écriture en mémoire long terme, pour éviter la saturation ?           |

### Synthèse Globale

**Jusqu'à la semaine 6** : Construction de la tuyauterie (pipeline, API, mémoire, tool use basique) — étape indispensable, car un agent sans bons outils ni bonne mémoire est inefficace.

**À la semaine 7** : LangGraph greffe le « cerveau agentique » (Reflection, Plan & Execute, Orchestrator/Worker) sur cette infrastructure. Le projet évolue ainsi d'un simple *moteur de recherche intelligent* à un véritable **Assistant de Recherche IA** autonome.